4. Write the python program for Cript-Arithmetic problem

Aim

To develop a Python program to solve the Crypt-Arithmetic Problem by assigning unique digits to letters such that the given arithmetic equation is satisfied.

Algorithm

1. Start the program.
2. Read the first word, second word, and result word.
3. Identify all unique letters.
4. Check whether the number of unique letters is less than or equal to 10.
5. Generate all possible digit assignments using permutations.
6. Ignore assignments where any leading letter is assigned zero.
7. Convert the words into numbers using the current assignment.
8. Check whether:
    - First Number + Second Number = Result Number.
9. If the equation is satisfied:
    - Display the letter-to-digit mapping.
    - Display the verified arithmetic expression.
    - Stop.
10. If all assignments are checked and no solution is found, display "No Solution Exists."
11. Stop.

Program

```
from itertools import permutations
# Input words
word1 = input("Enter First Word: ").upper()
word2 = input("Enter Second Word: ").upper()
result = input("Enter Result Word: ").upper()
# Get unique letters
letters = list(set(word1 + word2 + result))
# Check if more than 10 unique letters
if len(letters) > 10:
```

```
    print("Too many unique letters! Solution not possible.")
    exit()
# Leading letters cannot be zero
first_letters = {word1[0], word2[0], result[0]}
# Try all possible digit assignments
for perm in permutations(range(10), len(letters)):
    mapping = dict(zip(letters, perm))
    # Skip if leading letter is assigned zero
    if any(mapping[ch] == 0 for ch in first_letters):
        continue
    # Convert words to numbers
    num1 = int("".join(str(mapping[ch]) for ch in word1))
    num2 = int("".join(str(mapping[ch]) for ch in word2))
    num3 = int("".join(str(mapping[ch]) for ch in result))
    # Check solution
    if num1 + num2 == num3:
        print("\nSolution Found!\n")
        print("Letter Mapping:")
        for ch in sorted(mapping):
            print(f"{ch} = {mapping[ch]}")
        print("\nVerification:")
        print(f"{num1} + {num2} = {num3}")
        break
else:
    print("\nNo Solution Exists.")
```

Input

```
Enter First Word: SEND
Enter Second Word: MORE
Enter Result Word: MONEY
```

Output

Result

The Python program was successfully implemented to solve the Crypt-Arithmetic Problem using Permutation Search (Backtracking approach). The program accepts the two operand words and the result word from the user, assigns unique digits to each letter, verifies the arithmetic equation, and displays the correct letter-to-digit mapping when a valid solution is found. This demonstrates the application of constraint satisfaction techniques in Artificial Intelligence.